

Practical No: - 3

ON

Cryptography and Network Security - SSCS 4020

**TITLE: Implementation of Hill Cipher Encryption and Decryption
using Matrix Multiplication and Modular Arithmetic**

**BACHLEOR OF SCIENCE IN INFORMATION TECHNOLOGY
(HONOURS.)**

SUBMITTED BY:

Name: Patel Devan Sunil

Enrolment: 23SS02IT187

Class: BSCIT-H – 7A - Batch 2023-27

Objective

To implement the Hill Cipher encryption and decryption algorithm using matrix multiplication and modular arithmetic in Python.

Theory

The Hill Cipher is a classical polygraphic substitution cipher developed by **Lester S. Hill** in 1929. It is based on linear algebra and uses matrix multiplication to encrypt blocks of letters instead of encrypting one letter at a time.

INPUT : (hill_cipher.py)

```
key = [[3, 3],
        [2, 5]]

# Inverse Key Matrix (Modulo 26)
inverse_key = [[15, 17],
                [20, 9]]

# Convert text to numbers
def text_to_numbers(text):
    return [ord(ch) - ord('A') for ch in text]

# Convert numbers to text
def numbers_to_text(numbers):
    return ''.join(chr(num + ord('A')) for num in numbers)

# Prepare plaintext
def prepare_text(text):
    text = text.upper().replace(" ", "")
    if len(text) % 2 != 0:
        text += "X"
    return text

# Encryption Function
def encrypt(text):
    text = prepare_text(text)
    nums = text_to_numbers(text)
    result = []
```

```
for i in range(0, len(nums), 2):
    x = nums[i]
    y = nums[i + 1]
    c1 = (key[0][0] * x + key[0][1] * y) % 26
    c2 = (key[1][0] * x + key[1][1] * y) % 26
    result.extend([c1, c2])
return numbers_to_text(result)

# Decryption Function
def decrypt(cipher):
    nums = text_to_numbers(cipher)
    result = []
    for i in range(0, len(nums), 2):
        x = nums[i]
        y = nums[i + 1]
        p1 = (inverse_key[0][0] * x + inverse_key[0][1] * y) % 26
        p2 = (inverse_key[1][0] * x + inverse_key[1][1] * y) % 26
        result.extend([p1, p2])
    return numbers_to_text(result)

# Main Program
plaintext = input("Enter Plain Text: ")
ciphertext = encrypt(plaintext)
decrypted = decrypt(ciphertext)
print("\n----- RESULT -----")
print("Plain Text   :", prepare_text(plaintext))
print("Encrypted Text :", ciphertext)
print("Decrypted Text :", decrypted)
if prepare_text(plaintext) == decrypted:
    print("Verification : SUCCESS")
else:
    print("Verification : FAILED")
```

OUTPUT:

```

/usr/local/bin/python3 /usr/bin/devanpatel
● devanpatel@Devans-MacBook-Air Assignments
/SEM 7/Cyberspace/Assignments/p1.py"
Enter Plain Text: Devan

----- RESULT -----
Plain Text      : DEVANX
Encrypted Text   : VALQEL
Decrypted Text   : DEVANX
Verification    : SUCCESS

```

Student Task

Input:

```
key = [[3, 3],
```

```
       [2, 5]]
```

```
inverse = [[15, 17],
```

```
          [20, 9]]
```

```
text = "INFORMATION SECURITY"
```

```
# Remove spaces and convert to uppercase
```

```
text = text.upper().replace(" ", "")
```

```
# Add X if length is odd
```

```
if len(text) % 2 != 0:
```

```
    text += "X"
```

```
# Encryption
```

```
cipher = ""
```

```
for i in range(0, len(text), 2):
```

```
    a = ord(text[i]) - 65
```

```
    b = ord(text[i + 1]) - 65
```

```
    c1 = (key[0][0] * a + key[0][1] * b) % 26
```

```
    c2 = (key[1][0] * a + key[1][1] * b) % 26
```

```
    cipher += chr(c1 + 65)
```

```

cipher += chr(c2 + 65)

# Decryption
plain = ""

for i in range(0, len(cipher), 2):
    a = ord(cipher[i]) - 65
    b = ord(cipher[i + 1]) - 65
    p1 = (inverse[0][0] * a + inverse[0][1] * b) % 26
    p2 = (inverse[1][0] * a + inverse[1][1] * b) % 26
    plain += chr(p1 + 65)
    plain += chr(p2 + 65)

print("Original Plain Text :", "INFORMATION SECURITY")
print("Plain Text      :", text)
print("Encrypted Text   :", cipher)
print("Decrypted Text    :", plain)

if text == plain:
    print("Verification    : SUCCESS")
else:
    print("Verification    : FAILED")

```

OUTPUT:

```

● devanpatel@Devans-MacBook-Air Assignments % /usr/lo
/SEM 7/Cyberspace/Assignments/p2.py"
Original Plain Text : INFORMATION SECURITY
Plain Text          : INFORMATIONSECURITYX
Encrypted Text       : LDFCJQFROIPMSSHVDHLH
Decrypted Text       : INFORMATIONSECURITYX
Verification        : SUCCESS

```

Advantages and Disadvantages of Hill Cipher

Advantages

Encrypts multiple letters (blocks) at a time, making it stronger than simple substitution ciphers.

Uses matrix multiplication, making encryption more complex than monoalphabetic ciphers.

Disadvantages

The key matrix must be invertible modulo 26; otherwise, decryption is not possible.

Not secure against modern cryptanalysis and known-plaintext attacks.

Advantages

Provides better security than basic substitution ciphers.

Fast and easy to implement for small messages.

Helps in understanding matrix operations and modular arithmetic in cryptography.

Disadvantages

Key generation and matrix calculations are more complex than simple ciphers.

Requires padding (such as adding X) when the plaintext length is odd.

Mainly suitable for educational purposes and not recommended for real-world secure communication.

Viva Questions

Q1. What is Hill Cipher?

Answer: Hill Cipher is a polygraphic substitution cipher that uses matrix multiplication and modular arithmetic for encryption and decryption.

Q2. Which mathematical operation is used?

Answer: Matrix multiplication modulo 26.

Q3. Why is modulo 26 used?

Answer: Because the English alphabet contains 26 letters.

Q4. What is the key matrix?

Answer: The matrix used for encryption. In this practical:

[3 3]

[2 5]

Q5. Why is the inverse matrix required?

Answer: It is required to decrypt the ciphertext back into the original plaintext.

Q6. What happens if the plaintext length is odd?

Answer: An extra letter (usually X) is added to make the length even.